

DS Assignment 6

NAME: Prince Vyas

PRN: B24CE1062

/*Coffee Shop Line (Simple Queue):

Arrival: Customers arrive at the coffee shop and stand in line. **Order Processing:** The first customer in line gets their order taken, and the barista starts making the coffee. **Serving:** Once the first customer is served, they leave the queue, and the next customer in line moves forward to be served. Write a program to implement a simple queue

OR

Printer Spooler (Circular Queue):

In a multi-user environment, printers often use a circular queue to manage print jobs. Each print job is added to the queue, and the printer processes them in the order they arrive. Once a print job is completed, it moves out of the queue, and the next job is processed, efficiently managing the flow of print tasks. Implement the Printer Spooler system using a circular queue without using built-in queues.*/

PROGRAM:

```
#include <iostream>
using namespace std;
```

```
class shop {
    int size = 5;
    int token_shop[5];
    int r = -1; // rear
    int f = 0; // front
```

```
public:
    void Enqueue(int t);
    int Dequeue();
    int isEmpty();
    int isFull();
};
```

```
int shop::isEmpty() {
    return (f > r);
}
```

```

int shop::isFull() {
    return (r == size - 1);
}

void shop::Enqueue(int t) {
    if (isFull()) {
        cout << "\nQueue is full. Cannot issue token.";
    } else {
        r++;
        token_shop[r] = t;
        cout << "Token " << t << " is issued.";
    }
}

int shop::Dequeue() {
    if (isEmpty()) {
        cout << "\nQueue is empty. No orders to process.";
        return -1;
    } else {
        int x = token_shop[f];
        f++;
        cout << "\nToken " << x << " is processed.";
        return x;
    }
}

int main() {
    shop s;
    int t; // token no.
    int choice;

    do {
        cout << "\n--- Coffee Shop ---";
        cout << "\n1. Issue Token";
        cout << "\n2. Process Order";
        cout << "\n3. Exit";
        cout << "\nEnter your choice: ";
        cin >> choice;

        switch (choice) {

```

```
case 1:
    cout << "\nEnter token no.: ";
    cin >> t;
    s.Enqueue(t);
    break;

case 2:
    s.Dequeue();
    break;

case 3:
    cout << "\nExiting...";
    break;

default:
    cout << "\nInvalid choice. Please try again.";
    }
} while (choice != 3);

return 0;
}
```

```

--- Coffee Shop ---
1. Issue Token
2. Process Order
3. Exit
Enter your choice: 1

Enter token no.: 1
Token 1 is issued.
--- Coffee Shop ---
1. Issue Token
2. Process Order
3. Exit
Enter your choice: 2

Token 1 is processed.
--- Coffee Shop ---
1. Issue Token
2. Process Order
3. Exit
Enter your choice: 3

Exiting...

=== Code Execution Successful ===

```

2nd program:

/*Printer Spooler (Circular Queue):

In a multi-user environment, printers often use a circular queue to manage print jobs. Each print job is added to the queue, and the printer processes them in the order they arrive. Once a print job is completed, it moves out of the queue, and the next job is processed, efficiently managing the flow of print tasks. Implement the Printer Spooler system using a circular queue without using built-in queues.*/

PROGRAM:

```
#include <iostream>
```

```
using namespace std;
```

```
class PrinterSpooler {
    int jobs[5];    // Array to store print jobs
    int front;    // Points to the front job
    int rear;    // Points to the last job
    int size;    // Maximum size

```

public:

```
PrinterSpooler() {  
    size = 5;  
    front = -1;  
    rear = -1;  
}
```

```
bool isFull() {  
    return ((rear + 1) % size == front);  
}
```

```
bool isEmpty() {  
    return (front == -1);  
}
```

```
void addJob(int jobID) {  
    if (isFull()) {  
        cout << "\nQueue is full. Cannot add new print job.";  
        return;  
    }  
    if (isEmpty()) {  
        front = rear = 0;  
    } else {  
        rear = (rear + 1) % size;  
    }  
    jobs[rear] = jobID;  
    cout << "Print Job " << jobID << " added to the queue.\n";  
}
```

```
void processJob() {  
    if (isEmpty()) {  
        cout << "\nNo print jobs to process.";  
        return;  
    }  
    cout << "Processing Print Job: " << jobs[front] << endl;  
  
    if (front == rear) {  
        // Queue becomes empty  
        front = rear = -1;  
    }  
}
```

```

    } else {
        front = (front + 1) % size;
    }
}

void displayJobs() {
    if (isEmpty()) {
        cout << "\nNo print jobs in the queue.";
        return;
    }
    cout << "Current Print Jobs in Queue: ";
    int i = front;
    while (true) {
        cout << jobs[i] << " ";
        if (i == rear)
            break;
        i = (i + 1) % size;
    }
    cout << endl;
}
};

```

// Main function

```

int main() {
    PrinterSpooler spooler;
    int choice, jobID;

    do {
        cout << "\n--- Printer Spooler System ---";
        cout << "\n1. Add Print Job";
        cout << "\n2. Process Print Job";
        cout << "\n3. Display Queue";
        cout << "\n4. Exit";
        cout << "\nEnter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter Print Job ID: ";
                cin >> jobID;

```

```
        spooler.addJob(jobID);
        break;

    case 2:
        spooler.processJob();
        break;

    case 3:
        spooler.displayJobs();
        break;

    case 4:
        cout << "\nExiting Printer Spooler...";
        break;

    default:
        cout << "\nInvalid choice. Try again.";
    }
} while (choice != 4);

return 0;
}
```

OUTPUT:

Output

```
--- Printer Spooler System ---
1. Add Print Job
2. Process Print Job
3. Display Queue
4. Exit
Enter your choice: 1
Enter Print Job ID: 123
Print Job 123 added to the queue.
```

```
--- Printer Spooler System ---
1. Add Print Job
2. Process Print Job
3. Display Queue
4. Exit
Enter your choice: 2
Processing Print Job: 123
```

```
--- Printer Spooler System ---
1. Add Print Job
2. Process Print Job
3. Display Queue
4. Exit
Enter your choice: 3
```

```
No print jobs in the queue.
```

--- Printer Spooler System ---

1. Add Print Job

2. Process Print Job

3. Display Queue

4. Exit

Enter your choice: 4

Exiting Printer Spooler...

=== Code Execution Successful ===